

Juego de Baldosas

Barchin y Charos están jugando un juego en una cuadrícula de $2N \times 2M$ celdas cuadradas de tamaño unitario. Las filas están enumeradas de 0 a $2N - 1$ de arriba a abajo, y las columnas están numeradas de 0 a $2M - 1$ de izquierda a derecha. Para $0 \leq i < 2N$ y $0 \leq j < 2M$, denotamos la celda en la fila i y columna j como (i, j) .

Barchin le da a Charos $N \cdot M$ **bloques**, uno por uno. Cada bloque es un cuadrado de 2×2 que consiste de cuatro **baldosas** de 1×1 . Barchin ha coloreado cada baldosa del bloque de negro o blanco, garantizando que **al menos una baldosa es blanca**.

Charos debe colocar cada bloque en la cuadrícula **inmediatamente después de recibirla**, sin saber que bloque va a recibir ella después. Los bloques no pueden ser rotados. Cada bloque debe ser colocado completamente dentro de la cuadrícula, cubriendo exactamente cuatro celdas de la cuadrícula. Además, la casilla superior izquierda de cada bloque debe cubrir una celda cuyas coordenadas de fila y columna sean pares. Cada celda de la cuadrícula puede ser cubierta por a lo mucho un bloque.

Barchin gana el juego si, después de que cualquier bloque es colocado, existe un cuadrado de 2×2 celdas que esta cubierto por cuatro baldosas negras. Formalmente, si las celdas (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ están todas cubiertas por baldosas negras para algún $0 \leq a < 2N - 1$ y $0 \leq b < 2M - 1$, entonces gana Barchin. Los índices a y b no necesitan ser pares.

Charos gana si ella coloca todos los $N \cdot M$ bloques sin que Barchin gane. Note que colocar los $N \cdot M$ bloques cubrirá por completo la cuadrícula.

Tu tarea es implementar una estrategia para que Charos gane el juego. Se puede demostrar que, bajo las restricciones dadas, Charos siempre puede colocar los bloques de manera que garantice la victoria, independientemente del color de los bloques que reciba posteriormente.

Detalles de Implementación

Debes implementar dos procedimientos:

```
void init(int N, int M)
```

(**init** es abreviación de "inicializar")

- N : la mitad del número de filas en la cuadrícula.
- M : la mitad del número de columnas en la cuadrícula.
- La función es llamada exactamente una vez por caso de prueba, al comienzo de la ejecución de su programa.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

(**receive_block** significa "recibir bloque")

- TL, TR, BL, BR : los colores de las baldosas superior izquierda, superior derecha, inferior izquierda e inferior derecha del bloque actual, respectivamente, como se muestra en la imagen de abajo. Cada valor es 0 (blanco) o 1 (negro).
- Esta función es llamada exactamente $N \cdot M$ veces por caso de prueba, después de la llamada inicial a `init`.



Esta función debe retornar un par de enteros (i, j) , donde i es la coordenada de la fila y j es la coordenada de la columna de la celda donde se debe colocar la baldosa superior izquierda del bloque. Tanto i como j deben ser pares, y la región 2×2 cubierta por el bloque no debe superponerse a ningún bloque antes colocado.

Si `receive_block` devuelve en algún momento un par que no cumple con estas restricciones, o si después de colocar el bloque, un cuadrado de 2×2 celdas queda completamente cubierto por baldosas negras, el evaluador finalizará inmediatamente su programa y el veredicto para el caso de prueba será `Output isn't correct`.

El comportamiento del evaluador no es **adaptativo**. Esto significa que la secuencia de bloques que Barchin le da a Charos está fijada desde antes de llamar a `init`.

Retricciones

Para cada bloque, sea S el número de baldosas negras a lo largo de las cuatro baldosas. Es decir, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ para cada bloque.

Subtarea

Subtarea	Puntuación	Restricciones adicionales
1	6	$S = 1$ para cada bloque, y $N = 2$.
2	16	$S = 3$ para cada bloque. $N = M$, N es par, y cada uno de los cuatro posibles coloreos del bloque aparecen exactamente $\frac{N^2}{4}$ veces.
3	10	$S = 1$ para cada bloque
4	29	$S \leq 2$ para cada bloque.
5	39	Sin restricciones adicionales.

Ejemplo

Considere un juego con $N = 1$ y $M = 2$, entonces la cuadrícula tiene 2 filas y 4 columnas. El evaluador primero llama:

```
init(1, 2)
```

Inicialmente, todas las celdas están vacías. La cuadrícula se ve como:

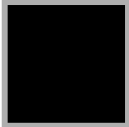
	0	1	2	3
0				
1				

Hay $N \cdot M = 2$ bloques para colocar. Suponga que Barchin le da un bloque con tres baldosas negras y una blanca en la esquina superior derecha. El evaluador llama:

```
receive_block(1, 0, 1, 1)
```

Charos decide colocar este bloque en el lado izquierdo de la cuadrícula retornando $(0, 0)$.

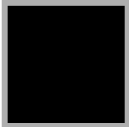
Ahora la cuadrícula se ve así:

	0	1	2	3
0				
1				

Barchin entonces le da otro bloque con tres baldosas negras y una baldosa blanca en la esquina superior izquierda:

```
receive_block(0, 1, 1, 1)
```

La única celda con fila par y columna par que puede funcionar como la esquina superior izquierda de un bloque de 2×2 es $(0, 2)$, entonces Charos retorna $(0, 2)$. La cuadrícula termina así:

	0	1	2	3
0				
1				

Ningun bloque de 2×2 está completamente cubierto por baldosas negras, entonces Charos ha colocado exitosamente todos los bloques sin que Barchin gane. Charos gana el juego.

Sample Grader (Evaluador de Ejemplos)

Formato de Entrada:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Formato de Salida:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Aquí, $R[k]$ y $C[k]$ es una pareja de enteros retornada por la k -th llamada de `receive_block`.